DG INTERNS HUB — TECHNICAL DOCUMENTATION

WEEK 4: SNORT IDS — LOCAL RULES CONFIGURATION & CUSTOM RULE WRITING

Submitted By: Gunjan
DG Interns Hub Cybersecurity
31 October 2026

TABLE OF CONTENTS

1. Introduction to Snort, IDS/IPS & Local Rules
2. Snort Rule Anatomy & Syntax Structure
3. Local Rules File Configuration
4. Practical Custom Snort Rules (5 Scenarios)
5. Understanding Snort Alert Telemetry
6. Rule Testing & Troubleshooting (Snort 2 vs 3)
7. MITRE ATT&CK Mapping (T1046)
8. SOC Analyst Operational Relevance
9. Common Syntax Mistakes & Best Practices
10. Conclusion & Documentation References

1. INTRODUCTION

What is Snort? Snort is an open-source Network Intrusion Detection System (NIDS) and Network Intrusion Prevention System (NIPS) created by Martin Roesch and currently maintained by Cisco Talos. It performs real-time traffic analysis and packet logging on IP networks.

IDS vs. IPS Mode: An *Intrusion Detection System (IDS)* operates out-of-band using TAP or SPAN port mirroring; it inspects traffic passively and generates alerts without affecting packet delivery. An *Intrusion Prevention System (IPS)* operates in-line; it actively inspects passing packets and can drop malicious traffic in real time.

Snort Local Rules & SOC Analyst Purpose: While vendor-supplied community rules detect known public exploits, local rules (`local.rules`) allow SOC analysts to write custom detection logic tailored to an organization's specific internal IP ranges, proprietary protocol endpoints, baseline anomaly parameters, and emergency zero-day Indicators of Compromise (IOCs).

2. SNORT RULE STRUCTURE & SYNTAX ANATOMY

A standard Snort rule is divided into two main parts: the **Rule Header** and the **Rule Options**.

```
action protocol source_ip source_port direction destination_ip destination_port (rule_options;)
```

Rule Header Components

- `action` : Tells Snort what to do when a packet matches (e.g., `alert` , `log` , `pass` , `drop` , `reject`).
- `protocol` : The protocol layer to inspect (e.g., `ip` , `icmp` , `tcp` , `udp`).
- `source_ip` / `destination_ip` : Defines source/destination addresses. Can be specific IPs (`192.168.1.50`), CIDR blocks (`10.0.0.0/24`), defined variables (`$HOME_NET` , `$EXTERNAL_NET`), or `any` . Negation is supported via `!` .
- `source_port` / `destination_port` : Specifies source/destination ports (e.g., `80` , `22` , ranges `1024:65535` , or `any`).
- `direction` : Traffic flow direction operator: directional (`->`) or bidirectional (`<>`).

Core Rule Options Key-Value Parameters

- `msg` : Text message printed in logs and SIEM alerts (e.g., `msg:"ICMP Echo Request Detected";`).
- `sid` : Snort ID (Unique Rule Identifier). Reserved ranges: `1-99,999` (Built-in/VRT rules), `100,000-999,999` (Community rules), `>= 1,000,000` (Custom local rules).
- `rev` : Revision number of the rule. Incremented whenever the rule logic is updated (e.g., `rev:1;`).
- `classtype` : Categorizes alerts for triage (e.g., `classtype:attempted-recon;`).
- `priority` : Severity rating override (1 = High, 2 = Medium, 3 = Low).
- `content` : Searches packet payload for explicit string or hex byte patterns (e.g., `content:"|90 90 90|";`). Options like `nocase` disable case sensitivity.

- `flags` : Checks for specific TCP control flags (e.g., `flags:S;` for SYN, `flags:F;` for FIN).
- `flow` : Specifies session state tracking (e.g., `flow:established,to_server;`).

3. LOCAL RULES FILE CONFIGURATION

The `local.rules` file is the designated location for custom analyst rules. Its location and inclusion directives depend on the Snort version installed on the host:

File Paths & Inclusion Directives

- **Snort 2:** Default location is typically `/etc/snort/rules/local.rules` . Included inside the main configuration file (`/etc/snort/snort.conf`) via the line:

```
include $RULE_PATH/local.rules
```

- **Snort 3:** Default location is typically `/usr/local/etc/snort/rules/local.rules` or `/etc/snort/rules/local.rules` . Included inside the Lua configuration file (`snort.lua`) via:

```
ips = { include = 'rules/local.rules' }
```

Validation & Syntax Verification

Before putting modified rules into live production monitoring, analysts validate syntax using Snort's test mode to prevent daemon crash loops:

- **Snort 2 Test Command:** `snort -T -c /etc/snort/snort.conf`
- **Snort 3 Test Command:** `snort -c /etc/snort/snort.lua -A dump --warn-all -s 65535 -T`

4. PRACTICAL CUSTOM SNORT LOCAL RULES

The following five custom rules illustrate practical threat detection patterns created for local network environments.

RULE 1: ICMP PING DETECTION

```
alert icmp $EXTERNAL_NET any -> $HOME_NET any (msg:"LOCAL-RULE ICMP Ping Request Detected"; it
ype:8; sid:1000001; rev:1;)
```

Technical Breakdown: Inspects inbound `icmp` traffic from external source networks to the home network. Option `itype:8` targets ICMP Echo Requests (ping). Assigned SID `1000001` to designate it as a custom local rule.

RULE 2: INCOMING SSH CONNECTION DETECTION

```
alert tcp $EXTERNAL_NET any -> $HOME_NET 22 (msg:"LOCAL-RULE Inbound SSH Connection Attempt";
flow:to_server,established; flags:S; sid:1000002; rev:1;)
```

Technical Breakdown: Monitors TCP packets directed at port 22 on home network hosts. `flags:S` targets the initial SYN packet of the TCP handshake, while `flow:to_server,established` tracks connection initiation to identify potential remote administration or brute-force connection attempts.

RULE 3: HTTP TRAFFIC DETECTION

```
alert tcp $HOME_NET any -> $EXTERNAL_NET 80 (msg:"LOCAL-RULE Unencrypted HTTP Web Request Outb
ound"; flow:to_server; content:"GET"; depth:3; sid:1000003; rev:1;)
```

Technical Breakdown: Inspects outbound web requests on port 80. `content:"GET"; depth:3;` verifies that the HTTP payload begins with the GET request method, enabling visibility into cleartext web browsing.

RULE 4: NMAP SYN SCAN RECONNAISSANCE DETECTION

```
alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"LOCAL-RULE Possible Nmap TCP SYN Scan Detected"; flags:S; window:1024; sid:1000004; rev:1; classtype:attempted-recon;)
```

Technical Breakdown: Nmap SYN scans send raw TCP SYN packets without completing the full three-way handshake. Default Nmap probes frequently utilize specific TCP window sizes (such as 1024). Matching `flags:S` combined with `window:1024` flags automated network scanning activity.

RULE 5: SUSPICIOUS OUTBOUND PASSWORD TRANSMISSION PATTERN

```
alert tcp $HOME_NET any -> $EXTERNAL_NET any (msg:"LOCAL-RULE Suspicious Cleartext Password Transmission"; content:"password="; nocase; sid:1000005; rev:1; classtype:suspicious-traffic;)
```

Technical Breakdown: Inspects outbound TCP payloads across all ports for cleartext credential strings (`password=`). The `nocase` modifier ensures case-insensitive matching. Useful for catching unencrypted form submissions or basic malware exfiltration attempts.

5. UNDERSTANDING SNORT ALERTS & TRIAGE

When a packet matches a rule header and options list, Snort generates a structured alert entry in `/var/log/snort/alert` or streams it to Syslog/SIEM log collectors:

```
[**] [1:1000004:1] LOCAL-RULE Possible Nmap TCP SYN Scan Detected [**]
[Priority: 0]
10/31-14:22:05.112045 198.51.100.45:49152 -> 10.0.0.15:80
TCP TTL:64 TOS:0x0 ID:54321 IpLen:20 DgmLen:40
******S* Seq: 0xA1B2C3D4 Ack: 0x0 Win: 0x400 TcpLen: 20
```

Alert Triage Workflow: When an alert fires, a Tier-1 SOC analyst checks:

1. **Rule SID & Priority:** Identifies which rule triggered and its severity rating.
2. **Source IP & Destination IP Context:** Checks threat intelligence feeds for the source IP and determines the business criticality of the target server.
3. **False Positive vs True Positive Evaluation:** Validates whether the packet represents legitimate administrative scanning (FP) or unauthorized external reconnaissance (TP).

6. RULE TESTING & EXECUTION COMMANDS

Testing rules requires running Snort against pre-recorded Packet Capture (PCAP) files or live network interfaces:

Snort 2 Commands

- **Test Rule File against PCAP:** `snort -c /etc/snort/snort.conf -r capture.pcap -A console`
- **Run Live Interface Monitoring:** `snort -c /etc/snort/snort.conf -i eth0 -A fast`

Snort 3 Commands

- **Test Rule File against PCAP:** `snort -c /etc/snort/snort.lua -r capture.pcap -A alert_fast`
- **Run Live Interface Monitoring:** `snort -c /etc/snort/snort.lua -i eth0 -A alert_fast`

7. MITRE ATT&CK MAPPING

Custom Snort rules map directly to the MITRE ATT&CK framework to provide standardized threat tracking:

- **Technique Mapping:** Rule 4 (Nmap SYN Scan) maps to **T1046 – Network Service Discovery** under the *Reconnaissance* and *Discovery* tactics.
- **Contextual Nuance:** Detecting a single SYN packet with window size 1024 indicates a technical packet pattern consistent with Nmap scanning tool defaults. However, network detection alone does not prove malicious intent—authorized internal vulnerability scanners produce identical network signatures. SOC analysts must cross-reference IP authorization lists before concluding an attack is underway.

8. COMMON SYNTAX MISTAKES & BEST PRACTICES

- **Invalid SID Values:** Never assign custom rules SIDs below `1,000,000`. Standard SIDs below 1,000,000 are reserved for Talos rulesets and cause duplicate ID crashes during update syncs.
- **Missing Semicolons:** Every option inside rule brackets must end with a semicolon (e.g., `msg:"Text"; sid:1000001;`). Missing semicolons cause parsing syntax errors during initialization.
- **Incorrect Rule Direction:** Using `<-` is invalid in Snort syntax. Use `->` for uni-directional or `<>` for bi-directional traffic flow.
- **Overly Broad Content Matching:** Searching short strings like `content:"GET";` without depth or offset constraints causes high CPU overhead and false positive floods.

9. CONCLUSION & REFERENCES

Custom Snort local rules provide SOC teams with granular control over network monitoring, allowing rapid deployment of zero-day detection signatures and internal policy compliance monitoring. Mastering Snort rule syntax is an indispensable skill for intrusion analysis and network threat hunting.

Official Documentation References:

- Snort 2 User Manual: https://www.snort.org/documents
- Snort 3 Documentation: https://docs.snort.org/